AWS EC2 and S3 Integration using IAM Roles

1. Project Overview

In this project, I used **Amazon Web Services** to build a simple cloud infrastructure using **Amazon EC2**, **Amazon S3**, and **AWS Identity and Access Management (IAM)**.

I launched an EC2 instance to act as a Linux server and connected to it remotely using SSH. After setting up the server, I created an S3 bucket to store files in the cloud. To allow the EC2 instance to interact with S3 securely, I attached an IAM role with the necessary permissions. This allowed the server to access AWS services without storing credentials directly on the machine.

Using the **AWS Command Line Interface**, I created a test file on the EC2 server and successfully uploaded it to the S3 bucket. Through this project, I learned how different AWS services work together and how IAM roles provide secure access between cloud resources.

2. Architecture

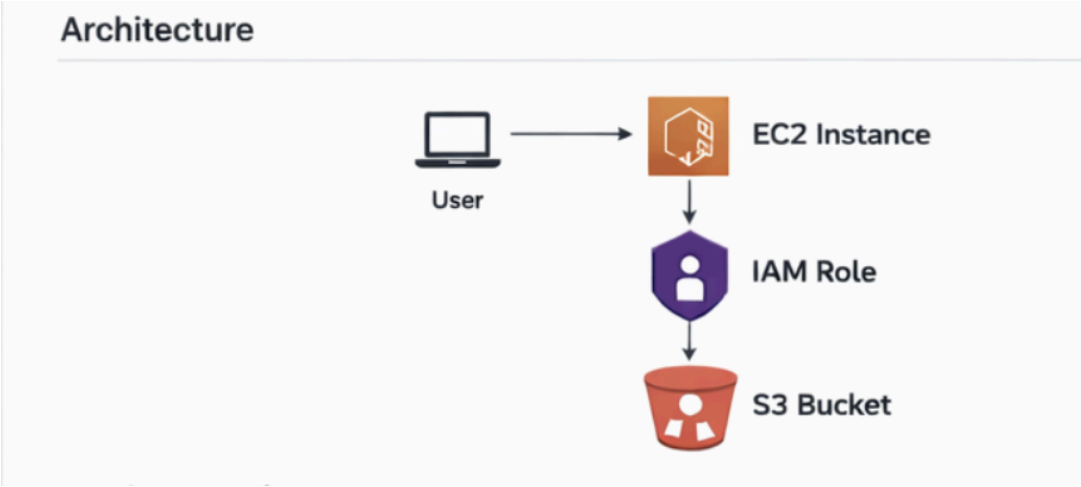


Figure 1: Architecture showing interaction between EC2, IAM Role, and S3

Architecture Components

- Amazon EC2 – Virtual Linux server used to run commands and create files.
- Amazon S3 – Cloud storage used to store files uploaded from the EC2 instance.
- AWS Identity and Access Management Role – Grants the EC2 instance permission to access the S3 bucket securely.

- Security Group – Controls network access to the EC2 instance.
- SSH – Used to remotely connect to the EC2 server.
- AWS Command Line Interface – Used to upload files from EC2 to S3.

3. Tools & Services Used

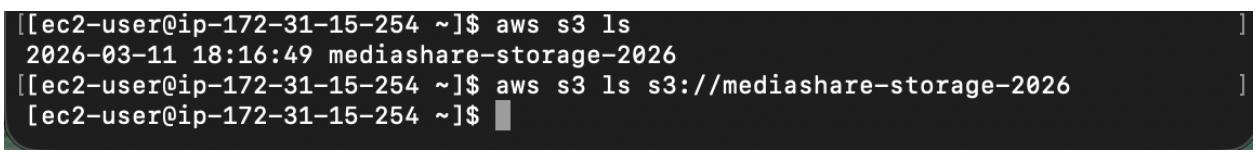
- **Cloud Platform**
  - Amazon Web Services
- **Services**
  1. Amazon EC2
  2. Amazon S3
  3. AWS IAM
  4. AWS CLI

4. Implementation Steps

- **Step 1: Configure AWS CLI**

  Configured the AWS CLI to securely interact with the AWS account from the terminal.

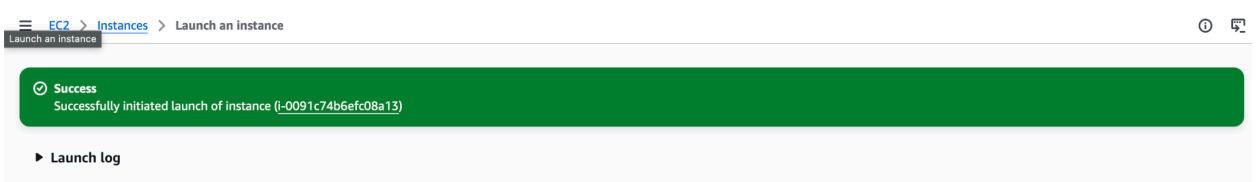
  Figure 2: CLI configuration

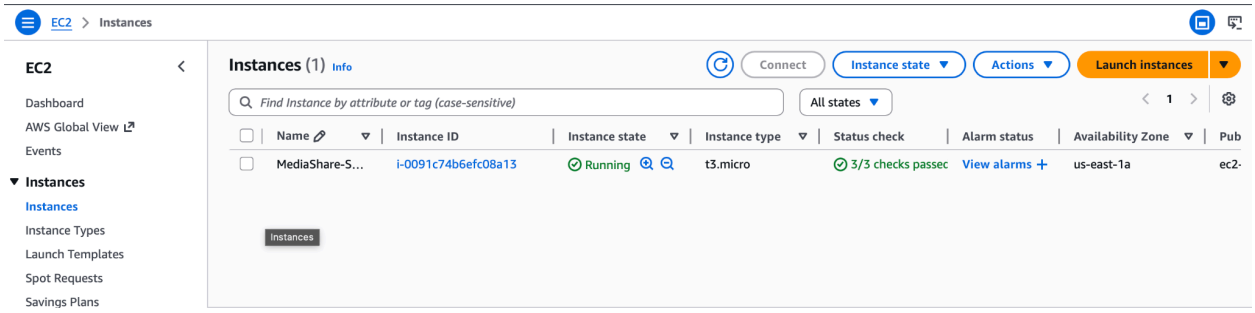


- **Step 2 — Launch EC2 Instance**

  Created an Amazon EC2 instance using the Free Tier and configured a security group to allow SSH access.

  Figure 3: EC2 instance running





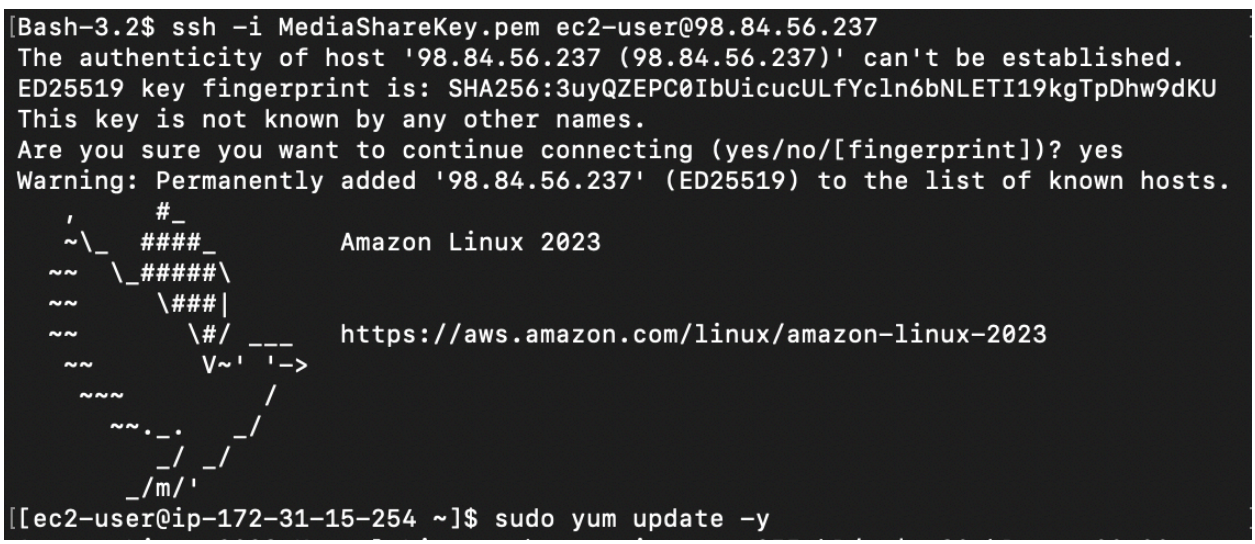
- **Step 3 — Connect to EC2 using SSH**

  Connected securely to the EC2 instance using a key pair.

  Command used:

  ssh -i MediaShareKey.pem ec2-user@public-ip

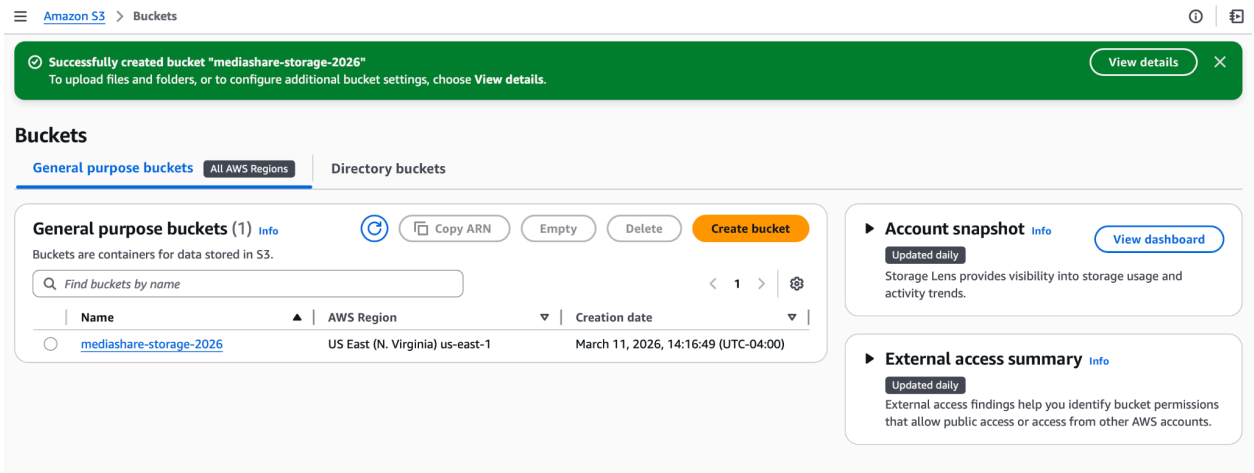
  Figure 4: SSH connection



- **Step 4 — Create S3 Bucket**

  Created an S3 bucket to store files generated by the EC2 instance.

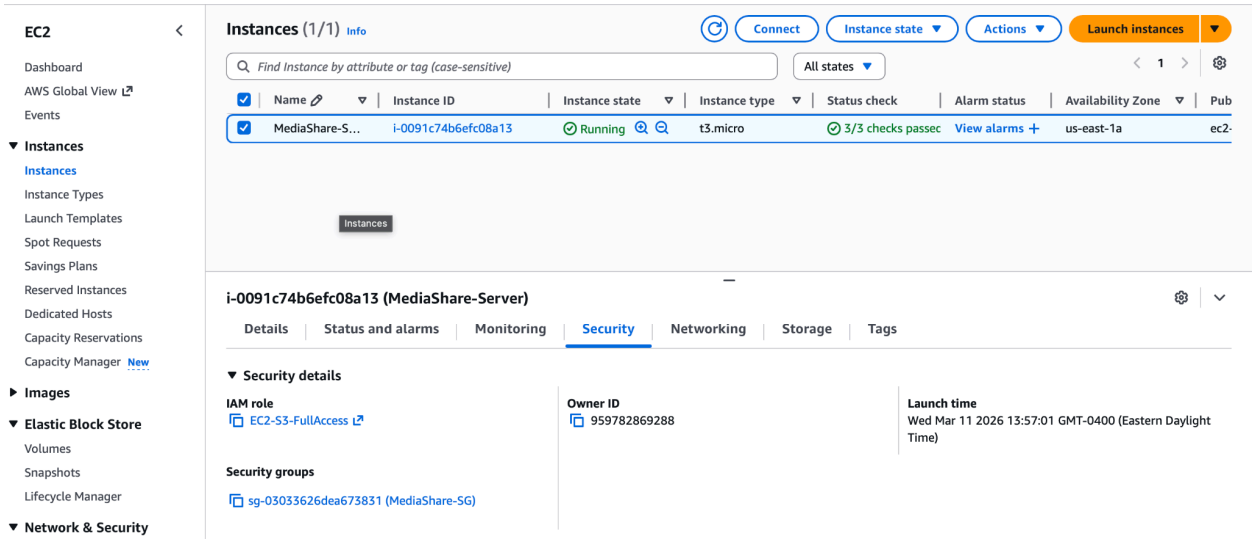
Figure 5: S3 bucket



- **Step 5 — Configure IAM Role**

Created and attached an IAM role to allow the EC2 instance to access S3 securely.

Figure 6: IAM role attached



- **Step 6 — Upload File from EC2 to S3**

Created a file and uploaded it using AWS CLI.

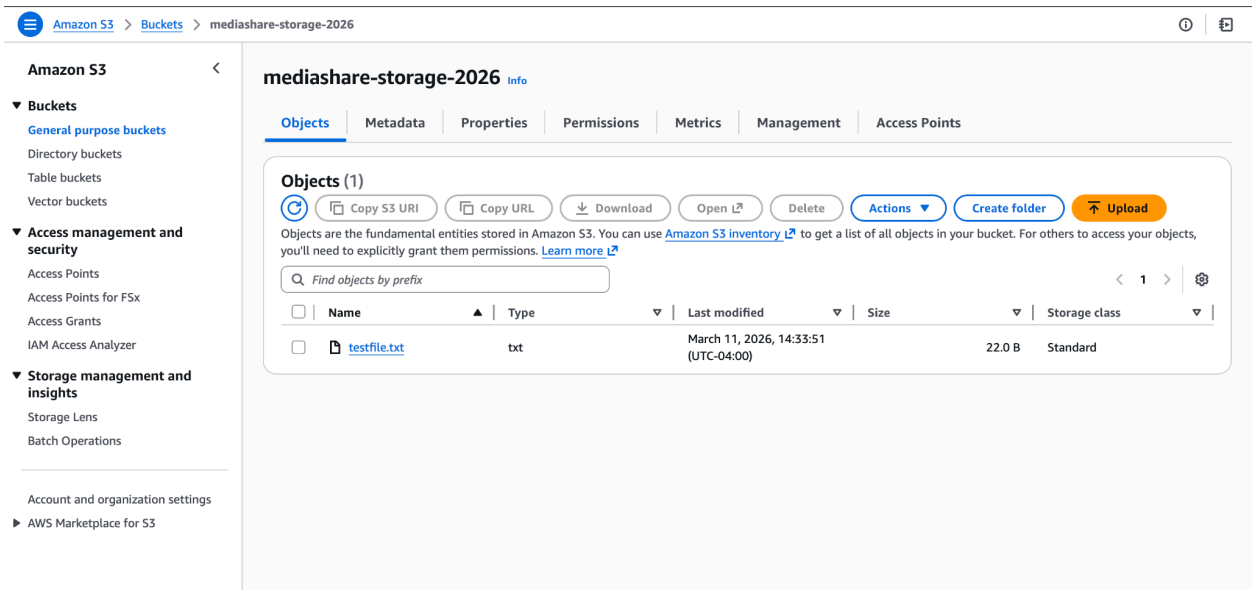
Command:

aws s3 cp testfile.txt s3://bucket-name

Figure 7: Successful file upload in terminal

```
[[ec2-user@ip-172-31-15-254 ~]$ echo "Hello from EC2 server" > testfile.txt      ]
[[ec2-user@ip-172-31-15-254 ~]$ aws s3 cp testfile.txt s3://mediashare-storage-20]
 26
 upload: ./testfile.txt to s3://mediashare-storage-2026/testfile.txt
 [ec2-user@ip-172-31-15-254 ~]$
```

Figure 8: Uploaded file visible in S3 bucket



5. Key Concepts Demonstrated

Through this project, I gained hands-on experience with several core cloud computing concepts using **Amazon Web Services**.

- **Cloud Infrastructure Deployment**
  Learned how to launch and configure a virtual server using **Amazon EC2**, which provides scalable compute resources in the cloud.
- **Secure Permissions with IAM Roles**
  Implemented **AWS Identity and Access Management** roles to grant the EC2 instance secure access to other AWS services without storing credentials directly on the server.
- **Remote Server Management with SSH**
  Connected to the EC2 instance from a local machine using SSH to manage and execute commands on the remote Linux server.

- **AWS CLI Usage**
  Used the **AWS Command Line Interface** to interact with AWS services and perform operations directly from the terminal.
- **Service Integration Between EC2 and S3**
  Established communication between **Amazon EC2** and **Amazon S3** by uploading files from the server to cloud storage using secure permissions.

6. Challenges & Solutions

SSH timeout: Updated the EC2 security group to allow inbound traffic on port 22.

S3 upload failed: Attached an IAM role with proper S3 write permissions.

AWS CLI credential error: Reconfigured the CLI to authenticate correctly with the AWS account.

7. Outcome

The project successfully demonstrated secure interaction between the **Amazon EC2** instance and **Amazon S3** using an **AWS Identity and Access Management** role. Files created on the EC2 server were uploaded to S3 using the **AWS Command Line Interface**, showing that the infrastructure worked as intended.

8. Future Improvements

Possible improvements:

- Automate infrastructure using Terraform
- Deploy a web application on EC2
- Implement CI/CD pipeline
- Add monitoring using CloudWatch

9. Conclusion

This project provided hands-on experience with core AWS services and demonstrated how secure service-to-service communication can be implemented using IAM roles. It also strengthened my understanding of cloud infrastructure deployment and command-line interactions with AWS services.